

STREAM ASSIGNMENT

PROBLEM STATEMENT:

Implement Snowflake Streams for Sales Data Processing

A retail company receives sales data in a staging table. The company wants to load this data into a final reporting table. After the initial load, only newly inserted or updated records should be processed using Snowflake Streams.

You need to create the database, tables, stream object, insert records, consume stream data, and update the final table using stream changes.

Task 1:

CREATE DATABASE

Create a transient database named:

STREAMS_DB

CODE:

create or replace transient database streams_db;

Task 2:

Create Raw Sales Staging Table

Create a table named:

SALES_RAW_STAGING

The table should contain the following columns:

id

product

price

amount

store_id

Use suitable data types.

CODE:

```
create or replace table sales_raw_staging
(
  id number,
  product string,
  price float,
  amount float,
  store_id number
);
```

Task 3:

Insert Initial Sales Data

Insert the following five records into SALES_RAW_STAGING:

id	product	price	amount	store_id
1	Banana	1.99	1	1
2	Lemon	0.99	1	1
3	Apple	1.79	1	2
4	Orange Juice		1.89	1
5	Cereals	5.98	2	1

CODE:

insert into sales_raw_staging values

(1,'banana',1.99,1,1),

(2,'lemon',0.99,1,1),

(3,'apple',1.79,1,2),

(4,'orange',1.89,1.89,1),

(5,'cereals',5.98,2,1);

SALES_RAW_STAGING TABLE

ID	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	banana	1.99	1	1
2	2	lemon	0.99	1	1
3	3	apple	1.79	1	1
4	4	orange	1.89	1.89	1
5	5	cereals	5.98	2	1

Task 4:

Create Store Table

Create a table named:

STORE_TABLE

The table should contain:

store_id

location

employees

CODE:

create or replace table store_table

(

store_id number,

location string,

employees number

);

Task 5:

Insert Store Data

Insert the following records into STORE_TABLE:

store_id	location	employees
1	Chicago	33

2

London

12

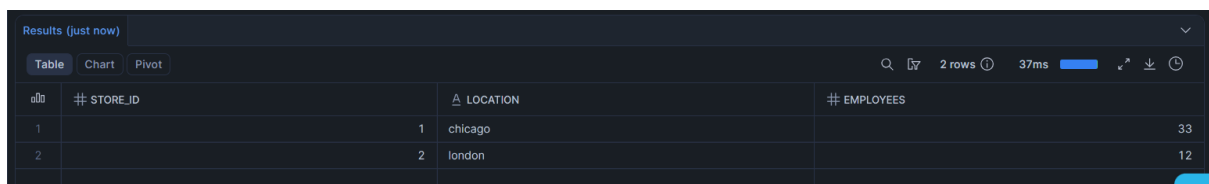
CODE:

insert into store_table values

(1,'chicago',33),

(2,'london',12);

STORE_TABLE



The screenshot shows a database query results window titled "Results (just now)". It displays a table with 2 rows. The table has four columns: an index column, "STORE_ID", "LOCATION", and "EMPLOYEES". The first row has values 1, 1, chicago, and 33. The second row has values 2, 2, london, and 12. The interface includes tabs for "Table", "Chart", and "Pivot", and a toolbar with search, filter, and export icons.

	STORE_ID	LOCATION	EMPLOYEES
1	1	chicago	33
2	2	london	12

Task 6:

Create Final Sales Table

Create a table named:

SALES_FINAL_TABLE

The table should contain:

id

product

price

amount

store_id

location

employees

This table will store sales data along with store location and employee details.

CODE:

create or replace table sales_final_table

as select

```
s.id,  
s.product,  
s.price,  
s.amount,  
s.store_id,  
st.location,  
st.employees  
from sales_raw_staging s join store_table st  
on s.store_id = st.store_id;
```

Task 7:

Perform Initial Full Load

Load data from SALES_RAW_STAGING into SALES_FINAL_TABLE.

While loading, join:

SALES_RAW_STAGING

with:

STORE_TABLE

using STORE_ID.

CODE:

```
create or replace table sales_final_table
```

```
as select
```

```
s.id,  
s.product,  
s.price,  
s.amount,  
s.store_id,  
st.location,
```

st.employees

from sales_raw_staging s join store_table st

on s.store_id = st.store_id;

Task 8:

Verify Initial Load

Display data from:

SALES_RAW_STAGING;

select * from sales_raw_staging;

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	banana	1.99	1	1
2	2	lemon	0.99	1	1
3	3	apple	1.79	1	1
4	4	orange	1.89	1.89	1
5	5	cereals	5.98	2	1

STORE_TABLE;

select * from store_table;

Results (just now)

TableChartPivot

🔍🔗2 rows37ms📄📧🕒

	# STORE_ID	LOCATION	# EMPLOYEES
1	1	chicago	33
2	2	london	12

SALES_FINAL_TABLE;

select * from sales_final_table;

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	banana	1.99	1	1	chicago	33
2	2	lemon	0.99	1	1	chicago	33
3	3	apple	1.79	1	2	london	12
4	4	orange	1.89	1.89	1	chicago	33
5	5	cereals	5.98	2	1	chicago	33

Confirm that SALES_FINAL_TABLE contains 5 records.

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
 Query produced no results							

Answer this question:

Why is SALES_STREAM empty immediately after creation?

ANSWER:

The sales_stream was empty because it track queries performed on the table after the stream was created on the table. The queries which was performed before creating stream was not tracked and no queries was performed on table after creating the stream.

This is the reason why stream was empty.

INSERT CHANGE UPDATE

Task 11:

Insert New Sales Records

Insert the following two records into SALES_RAW_STAGING:

id	product	price	amount	store_id
6	Mango	1.99	1	2
7	Garlic	0.99	1	1

CODE:

insert into sales_raw_staging values

(6,'mango',1.99,1,2),

(7,'garlic',0.99,1,1);

SALES_RAW_STAGING TABLE

#	# ID	△ PRODUCT	# PRICE	# AMOUNT	# STORE_ID
1		1 banana	1.99	1	1
2		2 lemon	0.99	1	1
3		3 apple	1.79	1	2
4		4 orange	1.89	1.89	1
5		5 cereals	5.98	2	1
6		6 mango	1.99	1	2
7		7 garlic	0.99	1	1

Task 12:

Check Stream Data

Query SALES_STREAM.

CODE:

```
select * from sales_stream;
```

Observe the new records and metadata columns:

METADATA\$ACTION

METADATA\$ISUPDATE

METADATA\$ROW_ID



	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	6	mango	1.99	1	2	INSERT	FALSE	2b4985978b936b80bf6b45b49bf31a448081161e
2	7	garlic	0.99	1	1	INSERT	FALSE	c29e009de39b4eb722ce56bebd8c193128ee9bbf

Answer this question:

What value do you see in METADATA\$ACTION for newly inserted records?

METADATA\$ACTION = INSERT

Task 13:

Verify Source and Target Tables

Check data in:

SALES_RAW_STAGING;

```
select * from sales_raw_staging;
```

#	# ID	△ PRODUCT	# PRICE	# AMOUNT	# STORE_ID
1		1 banana	1.99	1	1
2		2 lemon	0.99	1	1
3		3 apple	1.79	1	2
4		4 orange	1.89	1.89	1
5		5 cereals	5.98	2	1
6		6 mango	1.99	1	2
7		7 garlic	0.99	1	1

SALES_FINAL_TABLE;

select * from sales_final_table;

#	# ID	△ PRODUCT	# PRICE	# AMOUNT	# STORE_ID	△ LOCATION	# EMPLOYEES
1		1 banana	1.99	1	1	chicago	33
2		2 lemon	0.99	1	1	chicago	33
3		3 apple	1.79	1	2	london	12
4		4 orange	1.89	1.89	1	chicago	33
5		5 cereals	5.98	2	1	chicago	33

Answer this question:

Are the new records available in SALES_FINAL_TABLE before consuming the stream?

NO, THE SALES_FINAL_TABLE DOES NOT CONTAIN NEW DATA WHICH WAS INSERTED INTO THE TABLE SALES_RAW_STAGING

Task 14:

Consume Stream Data

SELECT * FROM SALES_STREAM

#	# ID	△ PRODUCT	# PRICE	# AMOUNT	# STORE_ID	△ METADATA\$ACTION	△ METADATA\$ISUPDATE	△ METADATA\$ROW_ID
1	6	mango	1.99	1	2	INSERT	FALSE	2b4985978b936b80bf6b45b49bf31a448081161e
2	7	garlic	0.99	1	1	INSERT	FALSE	c29e009de39b4eb722ce56bed8c193128ee9bbf

Insert new records from SALES_STREAM into SALES_FINAL_TABLE.

While inserting, join the stream with STORE_TABLE using STORE_ID.

CODE:

insert into sales_final_table

select

```
s.id,  
s.product,  
s.price,  
s.amount,  
s.store_id,  
st.location,  
st.employees  
from sales_stream s join store_table st  
on s.store_id = st.store_id;
```

```
insert into sales_final_table  
select  
s.id,  
s.product,  
s.price,  
s.amount,  
s.store_id,  
st.location,  
st.employees  
from sales_stream s join store_table st  
on s.store_id = st.store_id;
```

Results (just now)

Chart Pivot

1 row 904ms

number of rows inserted

2

Task 15:

Verify Stream After Consumption

Query:

```
SELECT * FROM SALES_STREAM;
```

Results (just now)

Table Chart Pivot

0 rows 404ms

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
----	---------	-------	--------	----------	------------------	--------------------	------------------

Query produced no results

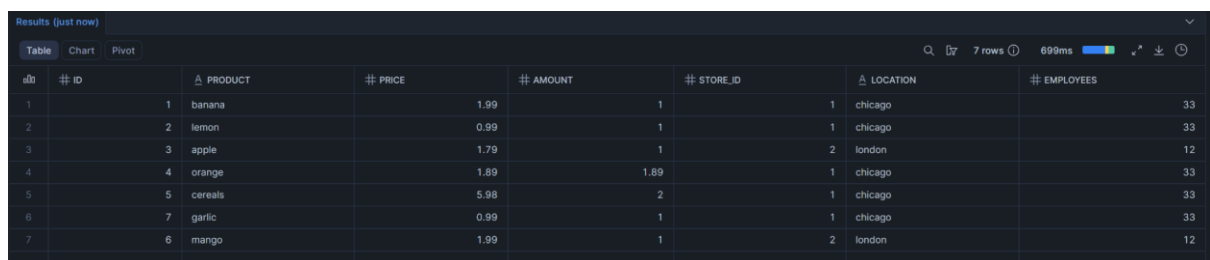
Answer this question:

Why did the stream become empty after inserting data into SALES_FINAL_TABLE?

ANSWER:

BECAUSE ,THE STREAM DATA WAS CONSUMED BY DML OPERATION (INSERT) WHICH WAS PERFORMED ON SALES_FINAL_TABLE.THE VALUES WHERE INSERTED TO SALES_FINAL_TABLE .

SALES_FINAL_TABLE



The screenshot shows a database query results window with the title 'Results (just now)'. It displays a table with 7 rows and 8 columns. The columns are: ID, # ID, Δ PRODUCT, # PRICE, # AMOUNT, # STORE_ID, Δ LOCATION, and # EMPLOYEES. The data rows are as follows:

	# ID	Δ PRODUCT	# PRICE	# AMOUNT	# STORE_ID	Δ LOCATION	# EMPLOYEES
1	1	banana	1.99	1	1	chicago	33
2	2	lemon	0.99	1	1	chicago	33
3	3	apple	1.79	1	2	london	12
4	4	orange	1.89	1.89	1	chicago	33
5	5	cereals	5.98	2	1	chicago	33
6	7	garlic	0.99	1	1	chicago	33
7	6	mango	1.99	1	2	london	12

SECOND INSERT CHANGE CAPTURE

Task 16:

Insert More Sales Records

Insert the following two records into SALES_RAW_STAGING:

id	product	price	amount	store_id
8	Paprika	4.99	1	2
9	Tomato	3.99	1	2

CODE:

insert into sales_raw_staging values

(8,'paprika',4.99,1,2),

(9,'tomato',3.99,1,2);

SALES_RAW_STAGING TABLE

Results (just now)

#	# ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	banana	1.99	1	1
2	2	lemon	0.99	1	1
3	3	apple	1.79	1	2
4	4	orange	1.89	1.89	1
5	5	cereals	5.98	2	1
6	6	mango	1.99	1	2
7	7	garlic	0.99	1	1
8	8	paprika	4.99	1	2
9	9	tomato	3.99	1	2

Task 17:

Consume Second Stream Data

SALES_STREAM

Results (just now)

#	# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	8	paprika	4.99	1	2	INSERT	FALSE	8efc2439a3e0129ab972110d4a105e593478bb96
2	9	tomato	3.99	1	2	INSERT	FALSE	7d389d3edb9241d8033b3ada7bd4f7c14cb94b4

Consume the newly captured stream records and insert them into SALES_FINAL_TABLE.

CODE:

```
insert into sales_final_table
```

```
select
```

```
s.id,
```

```
s.product,
```

```
s.price,
```

```
s.amount,
```

```
s.store_id,
```

```
st.location,
```

```
st.employees
```

```
from sales_stream s join store_table st
```

```
on s.store_id = st.store_id;
```

Task 18:

Verify Final Insert Output

Display data from:

SALES_FINAL_TABLE;

select * from sales_final_table;

115
116 | select * from sales_final_table;
117

Results (just now)

Table Chart Pivot

9 rows 76ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1		1 banana	1.99		1	1 chicago	33
2		2 lemon	0.99		1	1 chicago	33
3		3 apple	1.79		2	2 london	12
4		4 orange	1.89	1.89	1	1 chicago	33
5		5 cereals	5.98		2	1 chicago	33
6		7 garlic	0.99		1	1 chicago	33
7		6 mango	1.99		1	2 london	12
8		8 paprika	4.99		1	2 london	12
9		9 tomato	3.99		1	2 london	

SALES_RAW_STAGING;

select * from sales_raw_staging;

118
119 | select * from sales_raw_staging;
119

Results (just now)

Table Chart Pivot

9 rows 26ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	banana	1.99		1
2	2	lemon	0.99		1
3	3	apple	1.79		2
4	4	orange	1.89	1.89	1
5	5	cereals	5.98		2
6	6	mango	1.99		1
7	7	garlic	0.99		1
8	8	paprika	4.99		1
9	9	tomato	3.99		1

SALES_STREAM;

select * from sales_stream;

120
121 | select * from sales_stream;
121

Results (just now)

Table Chart Pivot

0 rows 371ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
---	----	---------	-------	--------	----------	------------------	--------------------	------------------

Query produced no results

How many records are now available in SALES_FINAL_TABLE?

ANSWER:

9 RECORDS ARE NOW AVAILABLE IN THE SALES_FINAL_TABLE

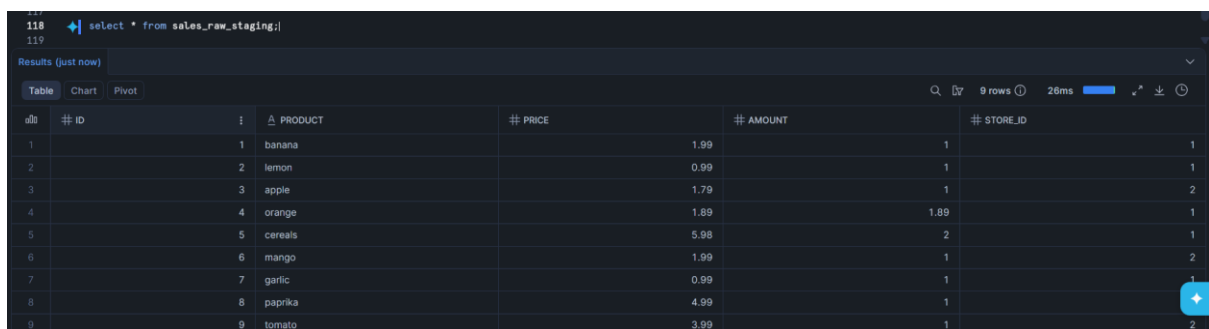
UPDATE CHANGE CAPTURE

Task 19:

Check Existing Data Before Update

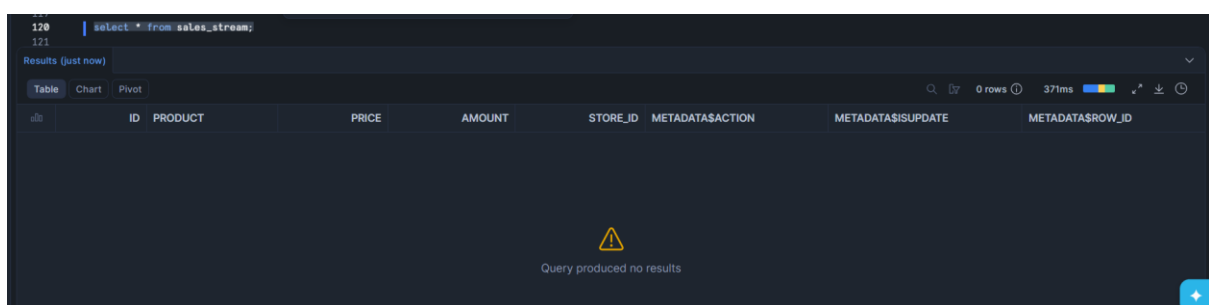
Display records from:

SALES_RAW_STAGING;



#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	banana	1.99	1	1
2	2	lemon	0.99	1	1
3	3	apple	1.79	1	2
4	4	orange	1.89	1.89	1
5	5	cereals	5.98	2	1
6	6	mango	1.99	1	2
7	7	garlic	0.99	1	1
8	8	paprika	4.99	1	1
9	9	tomato	3.99	1	2

SALES_STREAM;



ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
----	---------	-------	--------	----------	------------------	--------------------	------------------

Query produced no results

Task 20:

Perform First Update

Update the product name:

Banana

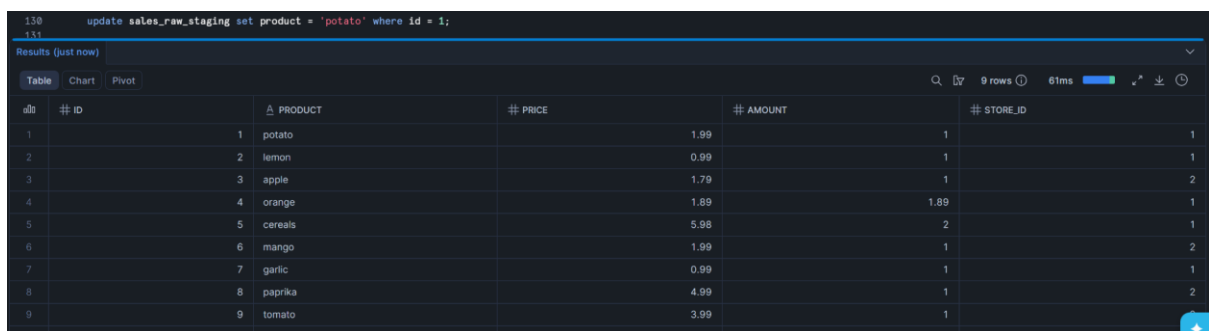
to:

Potato

in SALES_RAW_STAGING.

CODE:

`update sales_raw_staging set product = 'potato' where id = 1;`



The screenshot shows a SQL query execution interface. The query is `update sales_raw_staging set product = 'potato' where id = 1;`. The results are displayed in a table with 9 rows. The first row shows the update for id=1, where the product name is now 'potato'. The other rows show the original data for id=2 to id=9.

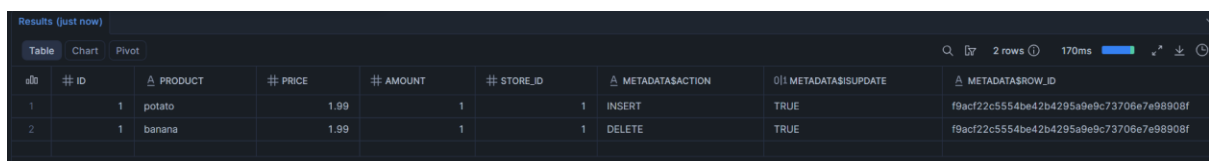
# ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	potato	1.99	1	1
2	lemon	0.99	1	1
3	apple	1.79	1	2
4	orange	1.89	1.89	1
5	cereals	5.98	2	1
6	mango	1.99	1	2
7	garlic	0.99	1	1
8	paprika	4.99	1	2
9	tomato	3.99	1	1

Task 21:

Check Stream After Update

Query:

`SELECT * FROM SALES_STREAM;`



The screenshot shows the results of the query `SELECT * FROM SALES_STREAM;`. The results are displayed in a table with 2 rows. The first row shows an INSERT operation for id=1, where the product name is 'potato'. The second row shows a DELETE operation for id=1, where the product name is 'banana'.

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	potato	1.99	1	1	INSERT	TRUE	f9acf22c5554be42b4295a9e9c73706e7e98908f
2	banana	1.99	1	1	DELETE	TRUE	f9acf22c5554be42b4295a9e9c73706e7e98908f

Answer:

How does Snowflake represent an UPDATE operation inside a stream?

UPDATE WILL BE SHOWN IN TWO RECORDS:

**1.DELETE (OLD ROW WILL BE DELETED) – UPDATES PERFORMED
ROW**

2.INSERT (THE UPDATED ROW WILL BE INSERTED) – NEW UPDATE ON THE ROW

Task 22:

Consume Update Using MERGE

Use a MERGE statement to update the corresponding record in SALES_FINAL_TABLE.

Condition:

Match records using ID

Update only records where METADATA\$ACTION = 'INSERT'

and METADATA\$ISUPDATE = TRUE

Update the following columns:

product

price

amount

store_id

CODE:

```
merge into sales_final_table f
```

```
using
```

```
(select
```

```
s.id,
```

```
s.product,
```

```
s.price,
```

```
s.amount,
```

```
s.store_id,
```

```
st.location,
```

```
st.employees
```

```
from sales_stream s join store_table st
```

on s.store_id = st.store_id where s.metadata\$action='INSERT')sk

on f.id = sk.id

when matched then update set

f.product=sk.product,

f.price=sk.price,

f.amount=sk.amount,

f.store_id=sk.store_id,

f.location=sk.location,

f.employees=sk.employees

WHEN NOT MATCHED THEN INSERT

(

id,

product,

price,

amount,

store_id,

location,

employees

)

VALUES

(

sk.id,

sk.product,

sk.price,

sk.amount,

```
sk.store_id,  
sk.location,  
sk.employees  
);
```

Task 23:

Verify First Update

Display data from:

SALES_FINAL_TABLE;

239 | select * from sales_final_table;
240

Results (just now)

Table Chart Pivot

9 rows 73ms

#	# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	potato	1.99	1	1	chicago	33
2	2	lemon	0.99	1	1	chicago	33
3	3	apple	1.79	1	2	london	12
4	4	orange	1.89	1.89	1	chicago	33
5	5	cereals	5.98	2	1	chicago	33
6	7	garlic	0.99	1	1	chicago	33
7	6	mango	1.99	1	2	london	12
8	8	paprika	4.99	1	2	london	12
9	9	tomato	3.99	1	2	london	12

SALES_RAW_STAGING;

130 | update sales_raw_staging set product = 'potato' where id = 1;
131

Results (just now)

Table Chart Pivot

9 rows 61ms

#	# ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	potato	1.99	1	1
2	2	lemon	0.99	1	1
3	3	apple	1.79	1	2
4	4	orange	1.89	1.89	1
5	5	cereals	5.98	2	1
6	6	mango	1.99	1	2
7	7	garlic	0.99	1	1
8	8	paprika	4.99	1	2
9	9	tomato	3.99	1	1

SALES_STREAM;

243 | select * from sales_stream;
244

Results (just now)

Table Chart Pivot

0 rows 1.2s

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
---	----	---------	-------	--------	----------	------------------	--------------------	------------------

Query produced no results

QUESTION:

Is Banana changed to Potato in SALES_FINAL_TABLE?

ANSWER

YES, BANANA WAS CHANGED TO POTATO

SECOND UPDATE CHANGE CAPTURE

Task 24:

Perform Second Update

Update the product name:

Apple

to:

Green apple

in SALES_RAW_STAGING.

CODE:

```
2 -- yes banana changed to potato -----  
3  
4 update sales_raw_staging set product = 'green apple' where id = 3;  
5
```

Task 25:

Consume Second Update Using MERGE

Use a MERGE statement to update the changed record in SALES_FINAL_TABLE.

CODE:

```
merge into sales_final_table f  
using  
(select  
s.id,
```

```
s.product,  
s.price,  
s.amount,  
s.store_id,  
st.location,  
st.employees  
from sales_stream s join store_table st  
on s.store_id = st.store_id where s.metadata$action='INSERT')sk  
on f.id = sk.id  
when matched then update set  
f.product=sk.product,  
f.price=sk.price,  
f.amount=sk.amount,  
f.store_id=sk.store_id,  
f.location=sk.location,  
f.employees=sk.employees
```

WHEN NOT MATCHED THEN INSERT

```
(  
id,  
product,  
price,  
amount,  
store_id,  
location,  
employees  
)  
VALUES  
(
```

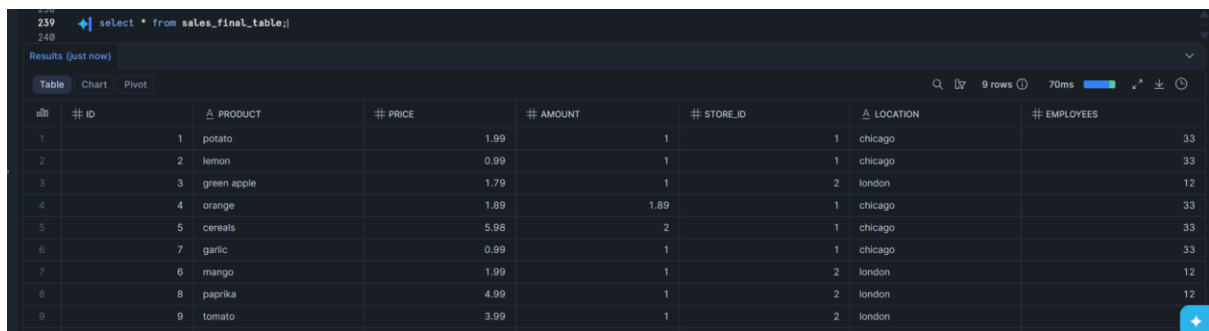
```
    sk.id,  
    sk.product,  
    sk.price,  
    sk.amount,  
    sk.store_id,  
    sk.location,  
    sk.employees  
);
```

Task 26:

Verify Final Output

Display data from:

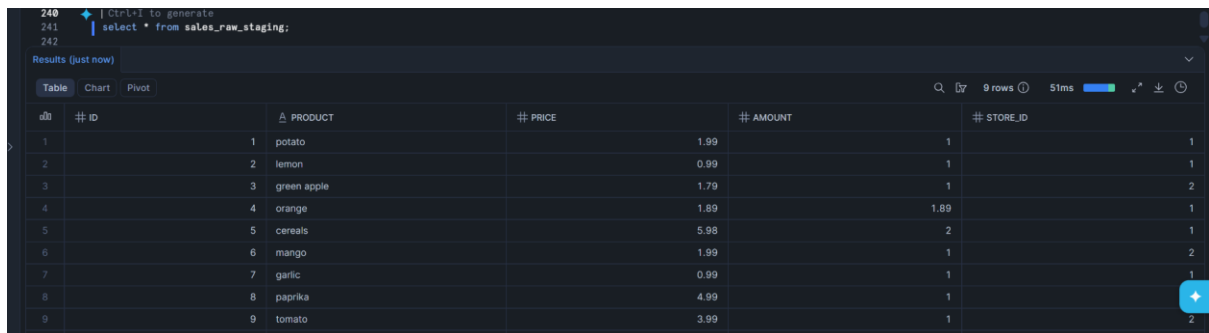
SALES_FINAL_TABLE;



The screenshot shows a SQL query result for the table `SALES_FINAL_TABLE`. The query is `select * from sales_final_table;`. The results are displayed in a table with 9 rows and 8 columns. The columns are: `# ID`, `PRODUCT`, `PRICE`, `AMOUNT`, `STORE_ID`, `LOCATION`, and `EMPLOYEES`. The data is as follows:

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	potato	1.99	1	1	chicago	33
2	lemon	0.99	1	1	chicago	33
3	green apple	1.79	1	2	london	12
4	orange	1.89	1.89	1	chicago	33
5	cereals	5.98	2	1	chicago	33
6	garlic	0.99	1	1	chicago	33
7	mango	1.99	1	2	london	12
8	paprika	4.99	1	2	london	12
9	tomato	3.99	1	2	london	12

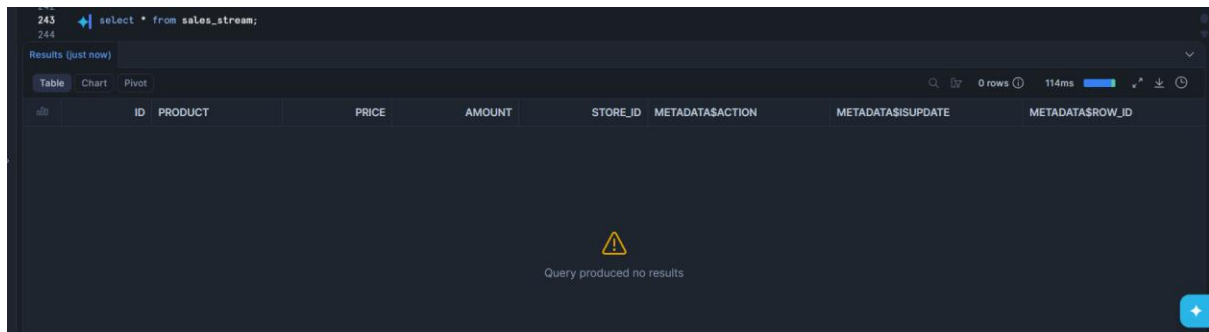
SALES_RAW_STAGING;



The screenshot shows a SQL query result for the table `SALES_RAW_STAGING`. The query is `select * from sales_raw_staging;`. The results are displayed in a table with 9 rows and 7 columns. The columns are: `# ID`, `PRODUCT`, `PRICE`, `AMOUNT`, `STORE_ID`, `LOCATION`, and `EMPLOYEES`. The data is as follows:

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	potato	1.99	1	1	chicago	33
2	lemon	0.99	1	1	chicago	33
3	green apple	1.79	1	2	london	12
4	orange	1.89	1.89	1	chicago	33
5	cereals	5.98	2	1	chicago	33
6	mango	1.99	1	2	london	12
7	garlic	0.99	1	1	chicago	33
8	paprika	4.99	1	2	london	12
9	tomato	3.99	1	2	london	12

SALES_STREAM;



Answer:

Is Apple changed to Green apple in SALES_FINAL_TABLE?

YES, THE APPLE WAS CHANGED TO GREEN APPLE